# Unit – 5

# Parallel Programming using OpenACC

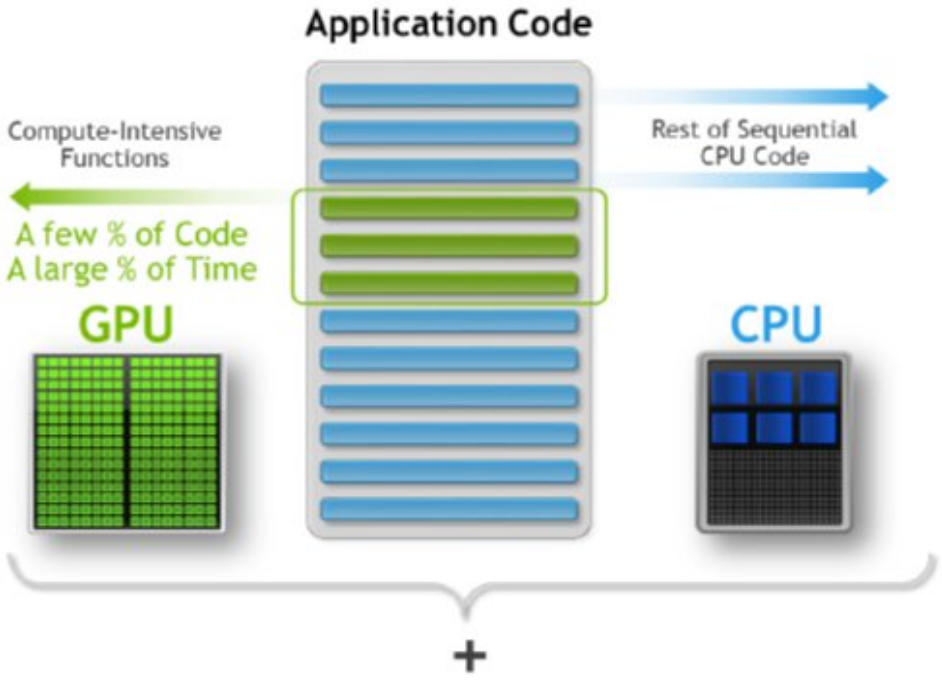
**By**

**Prof. Sandhya . S**

# OpenAcc

## Parallelize

- Insert OpenACC directives around important loops
- Enable OpenACC in the compiler
- Run on a parallel platform



# From serial to parallel programming using OpenACC

## A SIMPLE DATA-PARALLEL LOOP - #pragma acc parallel loop

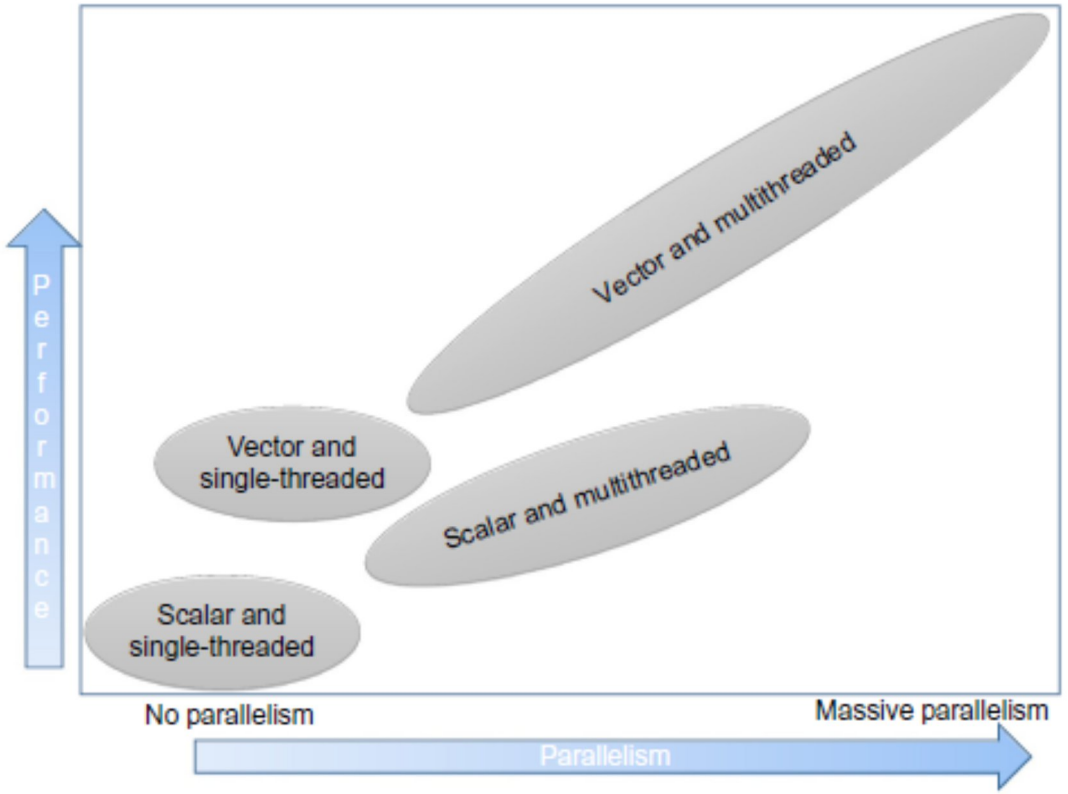


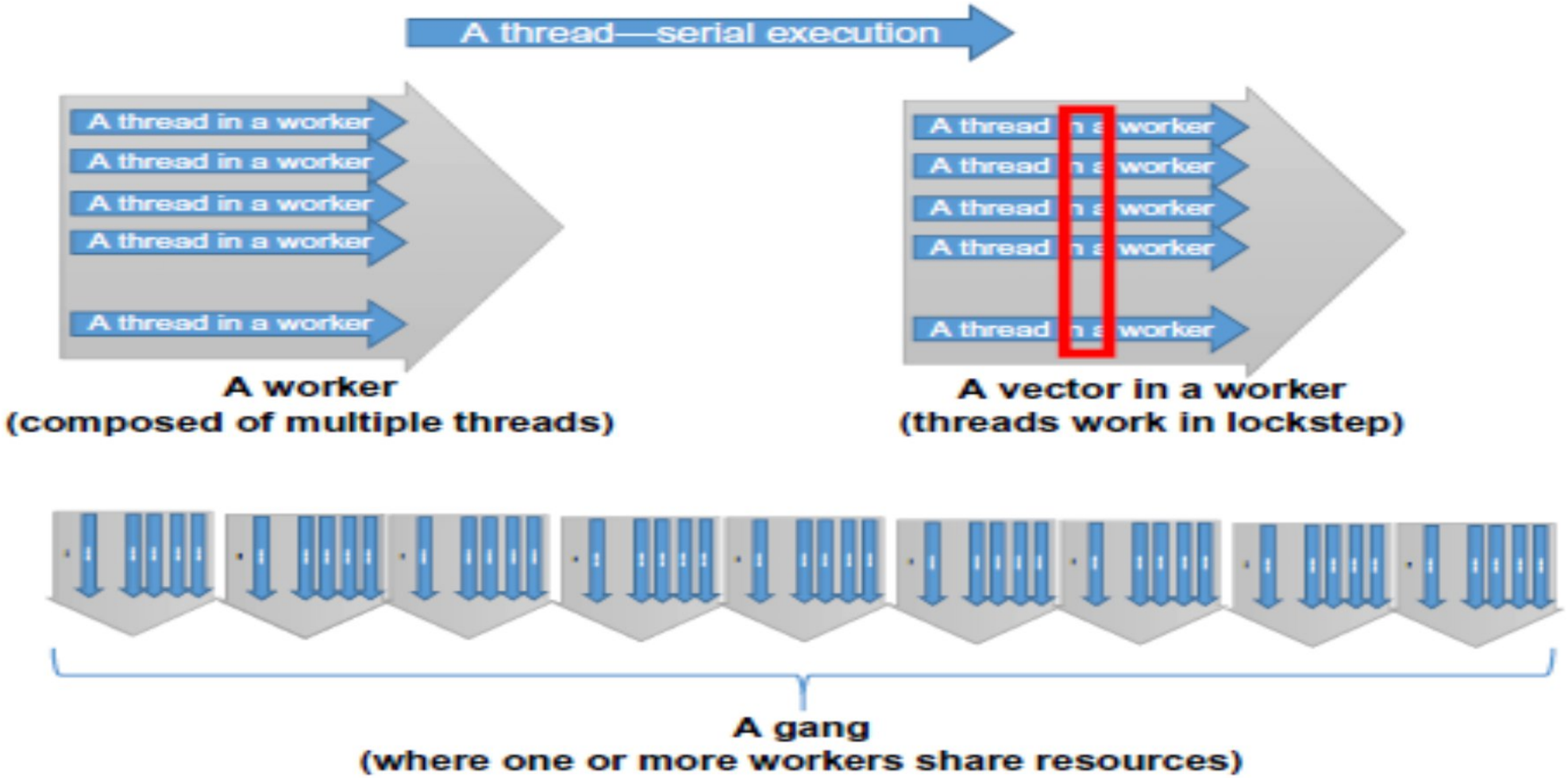

**Fig:** Multicore parallel and vector performance

# From serial to parallel programming using OpenACC

## THE VARIOUS FORMS OF OpenACC PARALLELISM:

- **A thread:** The core parallel concept is a single, serial thread of execution that runs any valid C, C++, or Fortran code.
- **A worker:** Groups of threads that can operate together in a SIMD or vector fashion are called workers.
- **Vectors:** Vectors cause worker threads to work in lockstep when running vector or SIMD instruction.
- **A gang:** Groups of workers are called gangs. Gangs operate independently of each other.

# THE VARIOUS FORMS OF OpenACC PARALLELISM:



# A SIMPLE TASK-PARALLEL EXAMPLE:

```
// a task that runs in a thread
#pragma acc routine worker
inline char task(int index, int nLoop)
{
  long counter=0;
  for(long i=0; i < 1000000*nLoop; i++)
      counter += (i>=0)?1:0;

  // return 1 if the counter is correct
  return( ((counter/1000000) == nLoop)?1:0 );
}
```

```
#include <iostream>
using namespace std;
#include <cstdlib>
#include <cassert>
#include <chrono>
using namespace std::chrono;

// a sequential task
// place task code here

int main(int argc, char *argv[])
{
  if(argc < 3) {
    cerr << "Use: nCount nLoop" << endl;
    return -1;
  }

  int nCount = atoi(argv[1]);
  int nLoop = atoi(argv[2]);

  if(nCount < 0 || nLoop < 0) {
    cerr << "ERROR: both nCount and nLoop must be greater than zero!" << endl;
    return -1;
  }

  high_resolution_clock::time_point t1 = high_resolution_clock::now();

  // Here is where we evaluate the task(s)
  int sum=0;
#pragma acc parallel loop reduction(+:sum)
  for(int i=0; i < nCount; i++)
    sum += task(i,nLoop);

  high_resolution_clock::time_point t2 = high_resolution_clock::now();
  duration<double> time_span = duration_cast< duration<double> >(t2 - t1);

  cout << "Duration " << time_span.count() << " second" << endl;
  cout << "Final sum is " << sum << endl;

  // Sanity check to see that array is filled with ones
  assert(sum == nCount);

  return 0; // normal exit
}
```

# AMDAHL'S LAW AND SCALING

- Named after the computer architect Gene Amdahl.
- It is an approximation that models the ideal speedup that can happen when serial programs are modified to run in parallel.

$$S(N) = \frac{1}{(1-P) + P/N}$$

- The expected speedup, $S(N)$
- $N$ processors
- Parallel program that $P$,
- portion of that cannot be parallelized, $(1 - P)$.

# BIG-O CONSIDERATIONS AND DATA TRANSFERS

Some common Big-O growth rates are as follows:

- O(1)
- O(N)
- $O(N^2)$

- BLAS (the Basic Linear Algebra Subprograms) library, BLAS is structured according to three different levels with increasing data and runtime requirements,-

i. **Level-1:** Vector-vector operations that require O(*N) data and O(N) work.*
ii. **Level-2:** Matrix-vector operations that require $O(N^2)$ *data and* $O(N^2)$ *work.*
iii. **Level-3:** Matrix-vector operations that require $O(N^2)$ *data and* $O(N^3)$ *work.*

# BIG-O CONSIDERATIONS AND DATA TRANSFERS

The three rules of offload mode programming:

i. *Get the data on the OpenACC device(s) and keep it there.*

*- present(), present_or_copy()*

i. *Give the OpenACC device(s) enough work to do.*

ii. *Focus on data reuse to avoid memory bandwidth limitations.*

# PARALLEL EXECUTION AND RACE CONDITIONS

- **OpenACC makes no guarantees about the order in which threads will run.**

```
#pragma acc parallel loop
 for(unsigned int i=0; i < nCount; i++) {
   // do something in each thread
 }
```

- **Race Conditions – will result due to data dependencies/shared variable.**

  Example: Multiple threads update a shared variable.

- **OpenACC provides**

  *#pragma acc atomic update* **to protect the shared variable.**

  *atomic read, write or update* **pragma's are provided by OpenACC to protect individual**

```
#include <iostream>
using namespace std;
#include <cstdlib>
#include <cassert>
#include <chrono>
using namespace std::chrono;

int main(int argc, char *argv[])
{
    if(argc < 2) {
       cerr << "Use: nCount" << endl;
       return 1;
       }
     int nCount = atoi(argv[1]);

    if(nCount <= 0) {
       cerr << "ERROR: nCount must be greater than zero!" << endl;
       return 1;
    }

   high_resolution_clock::time_point t1 = high_resolution_clock::now();

    // Here is where we define and increment the counter.
     int counter=0;
#pragma acc parallel loop
    for( int i=0; i < nCount; i++) {
#ifdef USE_ATOMIC
      #pragma acc atomic update
#endif
       counter++;
    }

    high_resolution_clock::time_point t2 = high_resolution_clock::now();
    duration<double> time_span = duration_cast<duration<double>>(t2 - t1);

    cout << "counter " << counter
         << " Duration " << time_span.count() << " second" << endl;
    assert(counter == nCount);

    return 0; // normal exit
}
```

# LOCK-FREE PROGRAMMING, CONTROLLING PARALLEL RESOURCES

## LOCK-FREE PROGRAMMING

- Mutual exclusion locks are a commonly used mechanism for synchronization.
- In OpenACC, locks are absent - *lock-free programming*.

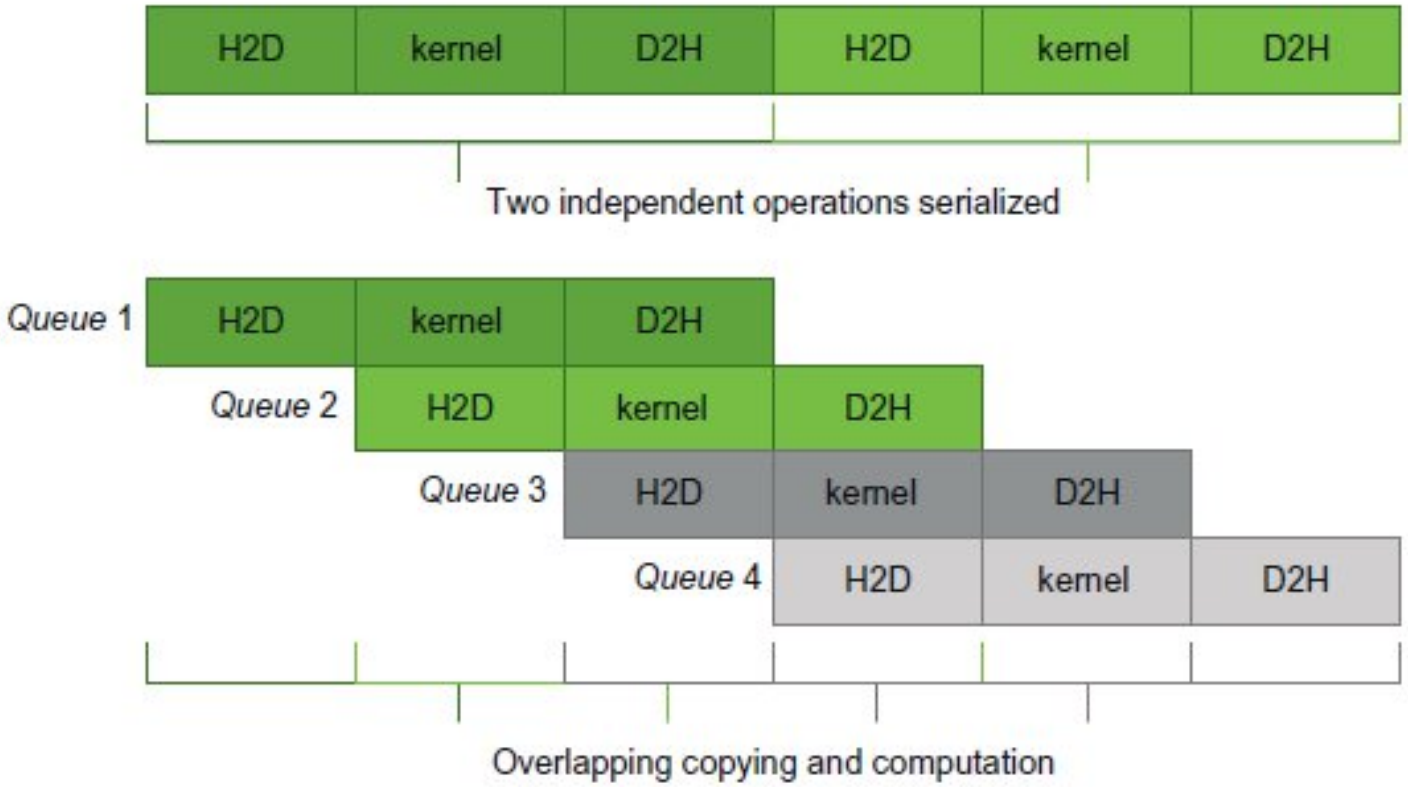
  In OpenMP #pragma omp critical - lock-*based programming*
  - *Does not* provide any of the performance guarantees
  - *Deadlock can occur*

## CONTROLLING PARALLEL RESOURCES

- GPU programmers speak in terms of *occupancy* - ***High occupancy.***
- *In* OpenACC - use a nested set of loops

# Pipelining data transfers with OpenACC

Figure: Serialize execution versus pipelined execution



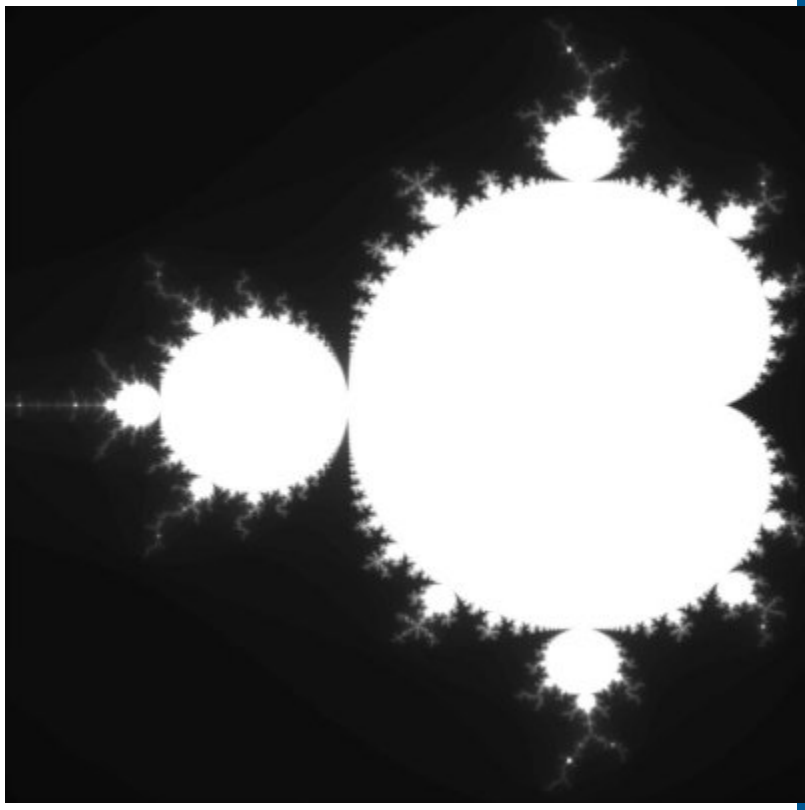
# EXAMPLE CODE: MANDELBROT GENERATOR

## MANDELBROT CODE

```
// Calculate value for a pixel
unsigned char mandelbrot(int Px, int Py) {
  double x0=xmin+Px*dx;   double y0=ymin+Py*dy;
  double x=0.0;   double y=0.0;
  for(int i=0;x*x+y*y<4.0 && i<MAX_ITERS;i++) {
    double xtemp=x*x-y*y+x0;
    y=2*x*y+y0;
    x=xtemp;
  }
  return (double)MAX_COLOR*i/MAX_ITERS;
}
 // Used in main()
 for(int y=0;y<HEIGHT;y++) {
    for(int x=0;x<WIDTH;x++) {
      image[y*WIDTH+x]=mandelbrot(x,y);
    }
  }
```

The mandelbrot() function calculates the color for each pixel.

Within main() there is a doubly-nested loop that calculates each pixel independently.



# EXAMPLE CODE: MANDELBROT GENERATOR

## MANDELBROT: ROUTINE DIRECTIVE

```
// mandelbrot.h
#pragma acc routine seq
unsigned char mandelbrot(int Px, int Py);

// Used in main()
#pragma acc parallel loop
for(int y=0;y<HEIGHT;y++) {
  for(int x=0;x<WIDTH;x++) {
    image[y*WIDTH+x]=mandelbrot(x,y);
  }
}
```

- At function source:
  - Function needs to be built for the GPU.
  - It will be called by each thread (sequentially)
- At call the compiler needs to know:
  - Function will be available on the GPU
  - It is a sequential routine

# Blocking Computation

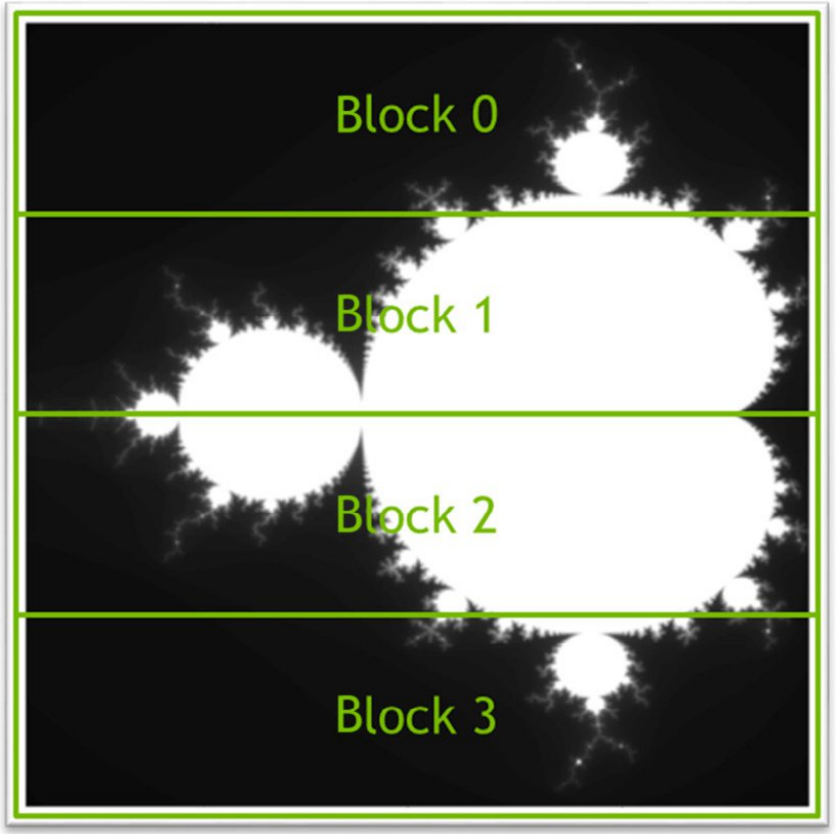
**PIPELINING MANDELBROT SET**

Steps

1.Break up computation into blocks along rows.

2.Break up copies according to blocks

3.Make both computation and copies asynchronous



# Blocking Computation

## PIPELINING MANDELBROT SET

```
22    num_blocks = 16;
23  #pragma acc data copyout(image[:WIDTH*HEIGHT])
24    for(int block = 0; block < num_blocks; block++ ) {
25      int start = block * (HEIGHT/num_blocks),
26          end   = start + (HEIGHT/num_blocks);
27  #pragma acc parallel loop
28      for(int y=start;y<end;y++) {
29        for(int x=0;x<WIDTH;x++) {
30          image[y*WIDTH+x]=mandelbrot(x,y);
31        }
32      }
33    }
```

```
22    num_blocks = 16;
23    block_size = (HEIGHT/num_blocks)*WIDTH;
24  #pragma acc data create(image[WIDTH*HEIGHT])
25    for(int block = 0; block < num_blocks; block++ ) {
26      int start = block * (HEIGHT/num_blocks),
27          end   = start + (HEIGHT/num_blocks);
28  #pragma acc parallel loop
29      for(int y=start;y<end;y++) {
30        for(int x=0;x<WIDTH;x++) {
31          image[y*WIDTH+x]=mandelbrot(x,y);
32        }
33      }
34  #pragma acc update self(image[block*block_size:block_size])
35    }
```